الجامعة الافتراضية السورية
SYRIAN VIRTUAL UNIVERSITY

TIC_IPG203_F25

البرمجة والتصميم غرضية التوجه

مدرس المادة الدكتور: غيث مصطفى

إعداد الطلاب

اسم الطالب	الـ ID	الصف
شام مجدي الحلواني	sham_356295	C1
ليث الدين حجازي	layth_aldiyn_292318	C1
عبد المجيد منار الدين القطيني	abd_almajeed_300872	C1
بيان عزات سليمان	bayan_259789	C1
نوار زياد حويجة	nawar_385545	C1

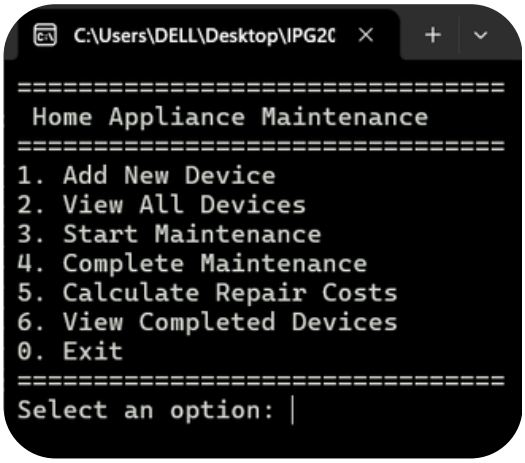
مقدمة المشروع

يهدف هذا المشروع إلى تطوير نظام برمجي لإدارة مركز صيانة وإصلاح الأجهزة المنزلية باستخدام لغة C#، مع تطبيق المبادئ الأساسية للبرمجة غرضية التوجه (OOP).
يوفر النظام إمكانية إدارة أنواع مختلفة من الأجهزة المنزلية، مثل الثلاجات والغسالات والمكيفات، ومتابعة حالتها أثناء عمليات الصيانة، وحساب تكلفة الإصلاح، بالإضافة إلى عرض الأجهزة التي اكتملت صيانتها.

تم تصميم البرنامج باستخدام مجموعة من الفئات والواجهات والعلاقات التي تعكس طبيعة العمل داخل مركز الصيانة، مع الاستفادة من مفاهيم التجريد والوراثة وتعدد الأشكال والتغليف والتفويض والأحداث.

فكرة عمل النظام

يعتمد النظام على قائمة تفاعلية تعمل من خلال واجهة Console، تسمح للمستخدم بتنفيذ العمليات الأساسية التالية:

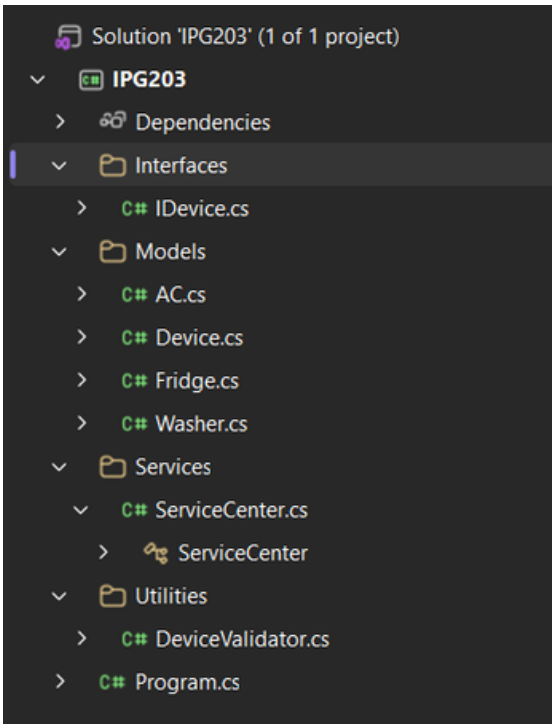


- إضافة جهاز جديد وتحديد نوعه.
- إدخال الرقم التسلسلي واسم الجهاز.
- عرض جميع الأجهزة المسجلة.
- بدء عملية الصيانة لجهاز محدد.
- إنهاء عملية الصيانة وتحديث حالة الجهاز.
- حساب تكلفة إصلاح الأجهزة.
- عرض الأجهزة التي اكتملت صيانتها.
- إظهار عدد الأجهزة المكتملة.

يتم تحديد الجهاز المطلوب من خلال رقمه التسلسلي، وتُحدّث حالته وفقاً للعملية التي ينفذها المستخدم.

الهيكل البرمجي للمشروع

تم تنظيم المشروع ضمن مجموعة من المجلدات لتسهيل إدارة الملفات وفصل المسؤوليات:



المحتويات	المجلد
الواجهات البرمجية	Interfaces
الفئات الأساسية والفئات المشتقة	Models
الفئة المسؤولة عن إدارة مركز الصيانة	Services
أدوات التحقق من صحة البيانات	Utilities
نقطة تشغيل البرنامج والقائمة التفاعلية	Program.cs

ويتكون النظام من العناصر البرمجية التالية:

- IDevice: واجهة العمليات المشتركة للأجهزة.
- Device: الفئة المجردة الأساسية.
- Fridge: فئة تمثل الثلاجة.
- Washer: فئة تمثل الغسالة.
- AC: فئة تمثل المكيف.
- ServiceCenter: فئة إدارة مركز الصيانة.
- DeviceValidator: فئة التحقق من صحة البيانات.
- StatusChangedHandler: التفويض المستخدم للإشعار بتغيير حالة الجهاز.

تطبيق مبادئ البرمجة غرضية التوجه

1. التجريد (Abstraction)

تم تطبيق مفهوم التجريد باستخدام الواجهة IDevice والفئة المجردة Device.
تحتوي الواجهة IDevice على العمليات المشتركة بين الأجهزة، وهي:

```
namespace IPG203.Interfaces
{
    1 reference
    internal interface IDevice
    {
        5 references
        void Start();
        5 references
        void Complete();
        3 references
        void Display();
    }
}
```

Start(): بدء عملية الصيانة.
Complete(): إنهاء عملية الصيانة.
Display(): عرض معلومات الجهاز.

أما الفئة المجردة Device فتطبق الواجهة السابقة، وتحتوي على الخصائص المشتركة للأجهزة، مثل الرقم التسلسلي والاسم والحالة والتكلفة.

كما تحتوي على الدالة المجردة:

CalcCost()

وهي مسؤولة عن حساب تكلفة الإصلاح، وتُترك طريقة تنفيذها للفئات المشتقة، بحيث تحسب كل فئة التكلفة المناسبة لنوع الجهاز.

2. الوراثة (Inheritance)

تم إنشاء ثلاث فئات مشتقة من الفئة الأساسية Device:

أولاً: Fridge
تمثل أجهزة الثلاجات، وتعيد تعريف الدوال المجردة الموروثة، وتحدد تكلفة الإصلاح بقيمة 150.

ثانياً: Washer
تمثل أجهزة الغسالات، وتعيد تعريف الدوال المجردة، وتحدد تكلفة الإصلاح بقيمة 200.

ثالثاً: AC
تمثل أجهزة التكييف، وتعيد تعريف الدوال المجردة، وتحدد تكلفة الإصلاح بقيمة 250.

تستخدم الفئات الثلاث الكلمة المفتاحية override لإعادة تعريف الدوال الموروثة، مع الاستفادة من الخصائص المشتركة الموجودة في الفئة الأساسية.

3. تعدد الأشكال (Polymorphism)

تم تطبيق تعدد الأشكال من خلال استخدام قائمة من النوع:

`List<Device>`

داخل الفئة ServiceCenter.

تستطيع هذه القائمة تخزين كائنات من الأنواع الثلاثة المشتقة، وهي Fridge وWasher وAC.

يظهر تعدد الأشكال بوضوح في الدالة:

`CalculateCosts()`

حيث تمر الدالة على جميع الأجهزة الموجودة في القائمة وتستدعي CalcCost() لكل جهاز.

وبما أن كل فئة مشتقة تمتلك تنفيذها الخاص لهذه الدالة، يتم حساب التكلفة وفقاً لنوع الجهاز الفعلي دون الحاجة إلى كتابة طريقة حساب منفصلة داخل مركز الصيانة لكل نوع.

4. التغليف (Encapsulation)

تم تطبيق التغليف داخل الفئة Device من خلال تعريف جميع الحقول على أنها private، وهي:

serial

name

status

cost



وتم توفير الخصائص (Properties) للتحكم بالوصول إلى هذه البيانات:

Serial: خاصية للقراءة فقط، مما يمنع تعديل الرقم التسلسلي بعد إنشاء الكائن.

Name: تسمح بقراءة اسم الجهاز وتعديله.

Status: تسمح بقراءة الحالة، بينما يقتصر تعديلها على الفئة الأساسية والفئات المشتقة من خلال protected set.

Cost: تسمح بقراءة التكلفة، بينما يقتصر تعديلها على الفئة الأساسية والفئات المشتقة.

كما تحتوي الفئة Device على Constructor يستقبل الرقم التسلسلي واسم الجهاز عند إنشاء الكائن، ويحدد الحالة الابتدائية للجهاز بالقيمة Pending.

يساعد هذا التصميم على حماية البيانات والتحكم بطريقة الوصول إليها وتعديلها.

5. التفويض والأحداث (Delegates & Events)

تم تعريف التفويض StatusChangedHandler لاستقبال قيمتين هما الرقم التسلسلي للجهاز والحالة الجديدة:

`StatusChangedHandler(string serial, string status)`

كما تم تعريف الحدث:

StatusChanged



داخل الفئة Device باستخدام هذا التفويض.

عند تغيير قيمة الخاصية Status يتم إطلاق الحدث باستخدام:

`StatusChanged?.Invoke(Serial, status);`

وفي الفئة ServiceCenter يتم الاشتراك بالحدث عند إضافة الجهاز إلى القائمة، من خلال ربطه بالدالة:

`OnStatusChanged()`

تقوم هذه الدالة بعرض رسالة توضح الرقم التسلسلي للجهاز والحالة الجديدة.

وبذلك يستطيع مركز الصيانة متابعة التغييرات التي تطرأ على حالة الأجهزة تلقائياً دون الحاجة إلى فحص حالة كل جهاز بشكل مستمر.

6. الفئات والأعضاء الثابتة (Static Classes & Members)

تم إنشاء الفئة الثابتة:

DeviceValidator

وتحتوي على دالتين ثابتتين للتحقق من صحة البيانات المدخلة:

IsValidSerial(): **تتحقق من أن الرقم التسلسلي ليس فارغاً أو مكوّناً من مسافات فقط.**

IsValidName(): **تتحقق من أن اسم الجهاز ليس فارغاً أو مكوّناً من مسافات فقط.**

كما تم تعريف الخاصية الثابتة:

CompletedCount

داخل الفئة Device، وهي مسؤولة عن تتبع عدد الأجهزة التي وصلت إلى حالة Completed.

تزداد قيمة الخاصية عند اكتمال صيانة الجهاز، وتنقص عند انتقال جهاز مكتمل إلى حالة أخرى.

وبذلك يمكن معرفة عدد الأجهزة المكتملة من خلال خاصية مشتركة بين جميع كائنات الأجهزة.

شرح الفئات الأساسية ووظائفها

1. الفئة Device

تمثل الفئة الأساسية المجردة لجميع الأجهزة، وتحتوي على البيانات والعمليات المشتركة، وتوفر البنية التي تعتمد عليها الفئات المشتقة.

2. الفئة ServiceCenter

تمثل مركز الصيانة، وتحتوي على قائمة الأجهزة، وتوفر الدوال التالية:



AddDevice(): **إضافة جهاز جديد وربطه بحدث تغيير الحالة.**

DisplayDevices(): **عرض جميع الأجهزة.**

CalculateCosts(): **حساب تكاليف الإصلاح.**

FindDevice(): **البحث عن جهاز باستخدام الرقم التسلسلي.**

DisplayCompletedDevices(): **عرض الأجهزة المكتملة.**

OnStatusChanged(): **استقبال إشعارات تغيير حالة الأجهزة.**

3. الفئة DeviceValidator

تتولى التحقق من صحة البيانات الأساسية قبل إنشاء الجهاز وإضافته إلى النظام.

حالات الجهاز

يعتمد النظام على ثلاث حالات أساسية:

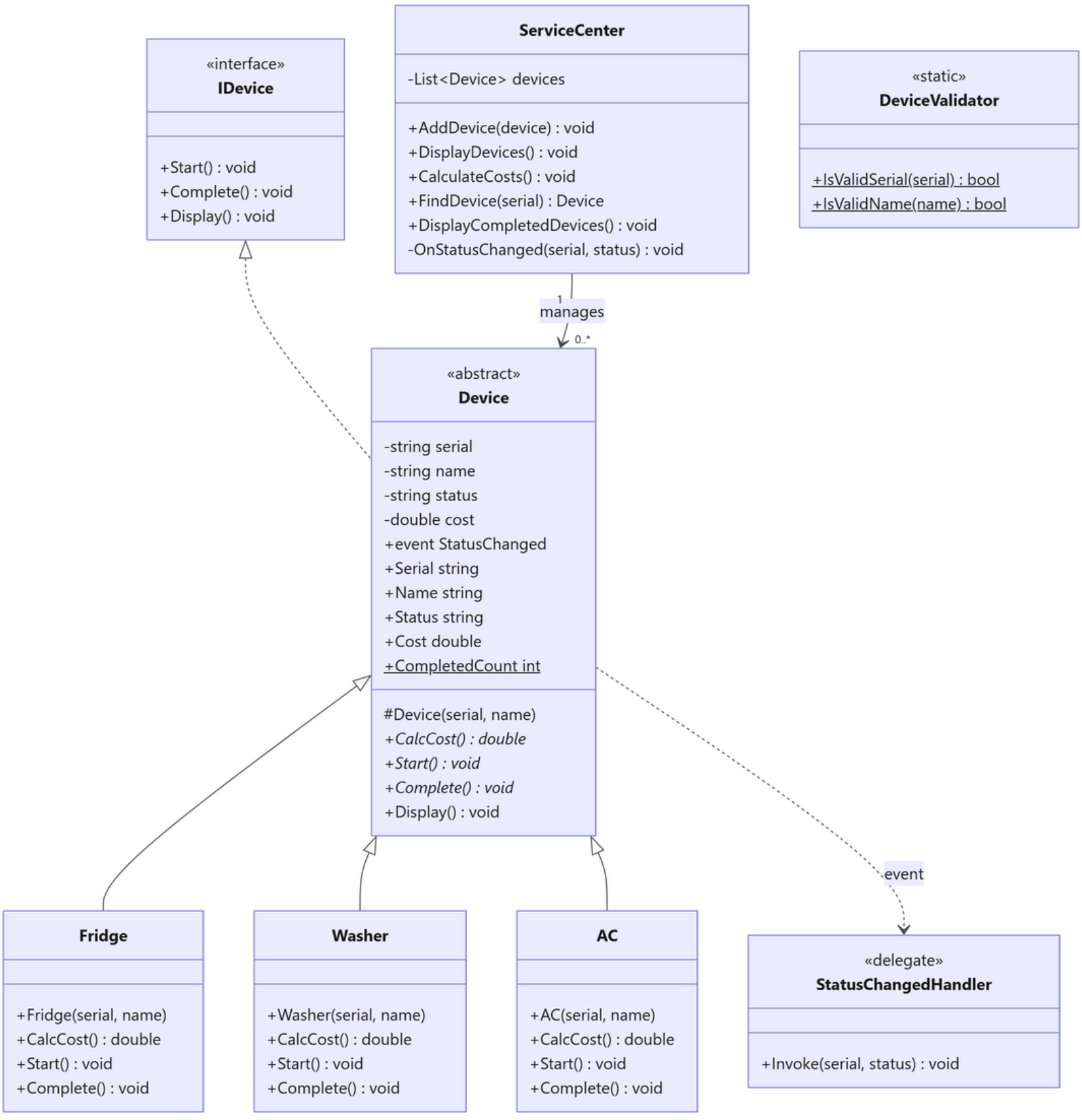
الوصف	الحالة
الجهاز بانتظار بدء الصيانة	Pending
الجهاز قيد الصيانة	In Progress
اكتملت صيانة الجهاز	Completed

تُحدد الحالة الابتدائية عند إنشاء الجهاز، وتتغير عند تنفيذ عمليات بدء الصيانة أو إنهائها.



مخطط UML

يوضح مخطط UML البنية العامة للنظام، والعلاقات بين الواجهة IDevice والفئة المجردة Device والفئات المشتقة منها، بالإضافة إلى علاقة مركز الصيانة بقائمة الأجهزة.



الاختبار والتحقق من عمل النظام

تم تشغيل البرنامج واختبار القائمة التفاعلية، والتحقق من تنفيذ العمليات الأساسية.
شملت الاختبارات:



- إضافة أجهزة من الأنواع المختلفة.
- عرض الأجهزة المسجلة.
- بدء صيانة جهاز باستخدام رقمه التسلسلي.
- إنهاء صيانة الجهاز وتحديث حالته.
- متابعة إشعارات تغيير الحالة.
- حساب تكاليف الإصلاح.
- عرض الأجهزة المكتملة وعددها.

وقد أظهر البرنامج إمكانية تنفيذ هذه العمليات من خلال القائمة التفاعلية.

الخاتمة

تم من خلال هذا المشروع تطوير نظام لإدارة مركز صيانة وإصلاح الأجهزة المنزلية باستخدام لغة C#، مع تطبيق المبادئ الأساسية للبرمجة غرضية التوجه بصورة عملية.

ساهم استخدام الواجهات والفئات المجردة والوراثة وتعدد الأشكال والتغليف في بناء تصميم منظم وقابل للتوسعة، كما ساعد استخدام التفويض والأحداث على متابعة تغييرات حالة الأجهزة، بينما وفرت الفئات والأعضاء الثابتة إمكانيات التحقق من البيانات وتتبع عدد الأجهزة المكتملة.

وقد أتاح المشروع تطبيق المفاهيم النظرية ضمن نظام يحاكي مجالاً عملياً، مع تنظيم الكود وفصل المسؤوليات بين مكونات البرنامج.



رابط المشروع على GitHub

رابط المستودع:

GitHub/IPG203: Object-Oriented Programming Project